

closure_evidence_20260923

BOB-203 closure evidence — 2026-09-23

Operator decision implemented

“Real auth middleware on 7186/7187” — wire BOBA_API_TOKEN into qbittorrent-proxy-go’s actual service and add genuine auth-enforcing middleware to both 7186 and 7187.

Root cause (confirmed independently)

Port 7186 (`plugins/download_proxy.py::DownloadHandler`, a raw stdlib HTTP handler that blindly reverse-proxies every request to qBittorrent) had ZERO awareness of BOBA_API_TOKEN anywhere — this was the real, live, exploitable defect the item measured. **Port 7187** (FastAPI, `download-proxy/src/api/*.py`) already had every mutating route protected via `Depends(require_api_token)` or a documented exemption by the time this item was worked — a prior, unrelated commit had already closed that half; the subagent’s own investigation found no 7187 code change was needed and confirmed this live (all four originally-cited 7187 routes independently re-measured returning 401 unauthenticated, not the originally-reported 422/404).

Fix

`plugins/download_proxy.py`: every mutating POST now requires BOBA_API_TOKEN (when armed) via `Authorization: Bearer Or X-Boba-Token`, constant-time compared (`hmac.compare_digest`), matching the existing `require_api_token` design exactly (same fail-open-when-unset policy, §11.4.122, unchanged) — except the two session-lifecycle

endpoints (/api/v2/auth/login, /api/v2/auth/logout), deliberately exempted since the WebUI's own browser login POST cannot attach a custom header, and exempting them closes no additional exposure (every actual mutation past login still requires the separate token). Also fixed a downstream header collision discovered during implementation: Authorization/X-Boba-Token are now stripped before forwarding to qBittorrent (qBittorrent's own server answers ANY request carrying an Authorization header with its own unrelated 403 regardless of session validity — without stripping, a caller correctly satisfying the new gate would then get rejected by qBittorrent itself).

docker-compose.yml: BOBA_API_TOKEN now also injected into qbittorrent-proxy-go (previously only download-proxy received it).

qBitTorrent-go/internal/middleware/apitoken.go (new): a Gin middleware mirroring require_api_token/the Python fix exactly, wired via a scoped route group covering only the routes with a Python-side Depends(require_api_token) counterpart — true route-level parity, not a blanket lockdown, matching the Python service's own public-by-design/known-gap route exemptions. This profile (--profile go) is opt-in and not running in this deployment, and is already loopback-bound by a prior unrelated operator decision (BOB-198) — this piece is defense-in-depth, not closing an active exposure; the subagent was explicit about this distinction and it is preserved here.

Findings on the two secondary investigation points

7185 (qBittorrent itself): confirmed real, but an ALREADY-KNOWN, ALREADY-DOCUMENTED, ALREADY-ACCEPTED risk (the hardcoded admin/admin WebUI credentials, per CLAUDE.md — explicitly “do not change”). qBittorrent enforces its own auth independently and is not bypassed by the proxy. No new code defect; out of this item's scope to alter.

7189 (boba-jackett) /api/v1/jackett/credentials: investigated and REFUTED. HandleListCredentials serializes only metadata booleans (has_username/has_password/has_cookies) — the struct's own comment states plaintext values are never serialized. No raw credential VALUES are returned by this or any other route. No new finding; nothing filed.

Independently re-verified this session (coordinator)

```
$ .venv/bin/python -m pytest tests/security/  
test_download_proxy_boba_token_auth.py -v --import-mode=importlib  
... 19 passed in 12.04s
```

```
$ cd qBitTorrent-go && go build ./... && go test ./internal/  
middleware/... -run "TestRED_UnsetToken|TestGREEN_SetToken|  
TestGREEN_EmptyBearerIsRejected|TestGREEN_TokenReadPerRequest" -v  
BUILD OK; 7/7 PASS
```

```
$ .venv/bin/python -m pytest tests/security/  
test_hooks_schedules_auth.py tests/unit/api_layer/  
test_download_token_auth.py tests/security/  
test_rate_limit_download_proxy.py -q --import-mode=importlib  
61 passed
```

Live, against the real running containers (both confirmed Up ... (healthy); the deployed download_proxy.py inside the container confirmed to genuinely contain the new _boba_token_ok function — the fix is actually deployed, not merely present in source):

```
$ curl -s -o /dev/null -w "%{http_code}" -X POST http://  
localhost:7186/api/v2/torrents/stop --data-urlencode hashes=0...0  
401    <- no auth at all: correctly refused before reaching  
qBittorrent
```

```
$ curl -s -o /dev/null -w "%{http_code}" -X POST http://  
localhost:7186/api/v2/torrents/stop -H "X-Boba-Token: <redacted>"  
--data-urlencode hashes=0...0  
403    <- correct token: passes the new gate, forwarded to  
qBittorrent,  
        which then correctly refuses for its OWN reason (no  
session) —  
        proving the fix lets a genuinely-authed request THROUGH,  
not  
        merely blocking everything
```

(Token value never printed in any command, output, or this evidence file — referenced only via an unprinted shell variable, per §11.4.10.)

Honest boundary (not silenced)

config/lan_route_auth_policy.yaml's exemption entries for the Go service's hooks/schedules/theme routes still show no auth marker for the new middleware — the subagent deliberately did not modify this

checked-in security-gate data file without being certain of the analyzer's exact resolution semantics for a `.Use()`-on-a-named-group idiom. Filed as its own small follow-up: **BOB-230** (see docs/qa/BOB-230/progress_note_20260923.md) — re-run scripts/pre_build/check_cm_lan_routes_authenticated.sh and update the policy file's qbittorrent-proxy-go.auth_markers if needed.

git diff –stat

```
docker-compose.yml | 11 ++++
plugins/download_proxy.py | 97 ++++++
+++++
qBittorrent-go/cmd/qbittorrent-proxy/main.go | 43 ++++++
3 files changed, 136 insertions(+), 15 deletions(-)
```

Plus new: qBittorrent-go/internal/middleware/apitoken.go,
qBittorrent-go/internal/middleware/apitoken_test.go, tests/
security/test_download_proxy_boba_token_auth.py.

Addendum — data-safety gate finding + fix (caught by commit-push-all.sh's

own pre_build_verification.sh sweep, invariant 55)

scripts/pre_build/check_cm_no_unscoped_live_destruction.sh
correctly flagged tests/security/
test_download_proxy_boba_token_auth.py's parametrized /api/v2/
torrents/delete test case as a static pattern that reads a torrent list
then deletes from it without diff-scoping — a legitimate concern in
general, since an unscoped delete against a REAL live instance would
destroy the operator's actual library.

Verified this specific case is safe: the test's `stack()` fixture spins up its
OWN throwaway `_StubQbtHandler` server bound to 127.0.0.1 on a
dynamically-allocated free port, and explicitly sets
`QBITTORRENT_HOST=127.0.0.1` so the proxy under test forwards to that
stub — never any real, configured, or live qBittorrent instance. Added
the project's own established in-source exemption marker (`# CM-NO-
UNSCOPED-LIVE-DESTRUCTION: EXEMPT reason=stub_server_not_a_client`),
matching the identical, already-accepted precedent in tests/unit/
test_stress_purge_scoping.py.

```
$ bash scripts/pre_build/check_cm_no_unscoped_live_destruction.sh
CM-NO-UNSCOPED-LIVE-DESTRUCTION: 3 exempted file(s):
```

```
    tests/pre_build/test_check_cm_no_unscoped_live_destruction.sh
[gate_fixture]
    tests/security/test_download_proxy_boba_token_auth.py
[stub_server_not_a_client]
    tests/unit/test_stress_purge_scoping.py
[stub_server_not_a_client]
CM-N0-UNSCOPED-LIVE-DESTRUCTION: PASS – 8 file(s) issue
destructive calls, all scoped

$ .venv/bin/python -m pytest tests/security/
test_download_proxy_boba_token_auth.py -q --import-mode=importlib
19 passed in 12.05s
```